

## PROYECTO DE PARALELIZACIÓN CON OPEN-MP: ECUACIÓN DE POISSON EN 2D

---

Asignatura: Sistemas Distribuidos

Docente: Carlos Andrés Gómez Vasco

Estudiantes: Juan Sebastian Manrique Moreno, Issabela del Pilar Ortiz Infante, Santiago Talero Parra

### Objetivo

Aplicar diferentes directivas de OpenMP para paralelizar un código secuencial que resuelve la ecuación de Poisson 2D mediante diferencias finitas, y analizar el impacto de cada estrategia en el rendimiento y estructura del programa.

### Preparación

1. Compila y ejecuta el código base secuencial.
2. Registra los resultados a continuación:

Tiempo de ejecución:  $7.893512 \times 10^{-1}$  segundos en total la ejecución completa.

Número de iteraciones: en promedio fueron aproximadamente 3322 iteraciones.

Tolerancia alcanzada: en promedio fue  $9.888511 \times 10^{-7}$ .

Número de threads (en este punto): 1

### Actividad 1: #pragma omp parallel for

Paralelizar el bucle principal en solve\_poisson.

Tareas:

- Aplica #pragma omp parallel for sobre el bucle de i.
- Usa reduction(max:delta) para evitar condiciones de carrera.

Resultados:

Número de threads usados: 28

Tiempo de ejecución:  $4.918907 \times 10^{-1}$  segundos en total la ejecución completa.

¿Hubo diferencia en el número de iteraciones? Sí, tuvo aproximadamente 3326.

- ¿Por qué es necesaria la cláusula reduction?

La cláusula reduction(max : delta) es necesaria porque delta es una variable compartida que se actualiza dentro de un bucle paralelo.

Sin reduction, múltiples hilos podrían leer y escribir delta al mismo tiempo, generando condiciones de carrera y resultados incorrectos.

- ¿Hubo mejora de rendimiento?

Sí, la mejora se puede evidenciar en los tiempos de ejecución que redujeron.

## **Actividad 2: collapse(2)**

Acelerar anidamiento de bucles.

Tareas:

- Sustituye el bucle paralelo anterior por:

```
#pragma omp parallel for collapse(2) reduction(max:delta)
```

Resultados:

Tiempo de ejecución:  $4.961067 \times 10^{-1}$  segundos en total la ejecución completa.

¿Mejoró respecto a la versión anterior? Sí / **No**

Reflexión:

- ¿Qué hace collapse(2)?

La cláusula collapse(2) en OpenMP fusiona dos bucles anidados (en este caso, los bucles i y j) en un solo bucle paralelo de mayor rango.

Esto permite que el planificador de OpenMP distribuya todas las iteraciones de ambos bucles como una única lista lineal de tareas entre los hilos.

- ¿Es siempre recomendable su uso?

No necesariamente, es recomendable cuando:

- Los bucles internos tienen igual o similar peso computacional.
- El número total de iteraciones es suficientemente grande.
- Se busca mejor distribución de carga entre hilos.

Y no es recomendable cuando:

- Hay dependencias entre bucles anidados (no independientes).
- El bucle externo ya tiene muchas iteraciones; collapse puede añadir sobrecarga innecesaria.
- La estructura de memoria no favorece el acceso lineal (podría aumentar el cache miss).

### Actividad 3: sections

Ejecutar funciones independientes en paralelo.

Tareas:

- Paraleliza initialize\_grid y poisson\_source con sections.

Resultados:

Tiempo de ejecución hasta terminar ambas funciones:

Al ejecutar por primera vez el tiempo fue de 0.0108217 segundos, sin embargo, al volverlo a ejecutar para las demás fuentes fue de  $1.86558 \times 10^{-3}$  en promedio.

¿Tuviste que reordenar las funciones? ¿Por qué?

No, debido a que en el código serial se lleva un orden de ejecución estricto entre las dos funciones, a la hora de paralelizar se rompe este orden estricto haciendo que no sea necesario reorganizar las funciones ya que no dependen una de la otra explícitamente.

Reflexión:

- ¿Vale la pena paralelizar funciones tan rápidas?

No, de hecho, en las ejecuciones que se realizaron el tiempo total de ejecución fue mayor a los tiempos de las anteriores actividades; este fue de  $5.278493 \times 10^{-1}$  segundos.

### Actividad 4: parallel y for separados

Controlar la región paralela.

Tareas:

- Usa:

```
#pragma omp parallel
{
    #pragma omp for reduction(max:delta)
}
```

- Añade variantes con schedule(static) y schedule(dynamic).

Resultados de tiempo:

Estrategia:

- Serial: 1.14042 segundos
- schedule(static): 1.09616 segundos.
- schedule(dynamic): 0.780539 segundos

## **Actividad 5: nowait, barrier, single, critical**

Control de sincronización.

Tareas:

- Usa nowait en bucles independientes.
- Imprime inicio de cada iteración con `#pragma omp single`.
- Agrega sección artificial crítica con `#pragma omp critical`.

Resultados:

Tiempo con nowait: 88.36 segundos, con 8 hilos, una malla

**¿Hubo cambio con respecto al uso por defecto?**

No, no produjo un cambio significativo en el tiempo de ejecución en este caso. Esto se debe a que el trabajo restante fuera del bucle paralelo no era pesado y los hilos de todas formas debían sincronizarse posteriormente con barrier. Sin embargo, en otros escenarios donde hay múltiples regiones independientes, nowait puede evitar esperas innecesarias y mejorar el rendimiento.

**¿Dónde colocaste la directiva single y por qué?**

Colocamos la directiva `#pragma omp single` justo al inicio de cada iteración dentro de la región paralela. Esto nos asegura que solo un hilo imprima el mensaje de inicio de iteración. Usar single evita que todos los hilos accedan simultáneamente a la salida estándar (`std::cout`), lo cual puede causar condiciones de carrera o ralentizar el programa.}

**¿critical ralentizó la ejecución? ¿En qué contexto sería útil?**

Sí, critical puede ralentizar la ejecución si se usa en una sección donde muchos hilos compiten por entrar, como ocurre con `std::cout`. La sección crítica fue artificial y usada solo con fines demostrativos. En contextos reales, `#pragma omp critical` es útil cuando varios hilos necesitan modificar o acceder a una variable compartida no protegida, por ejemplo, una estructura de datos o un archivo compartido.

**Actividad Final: Comparación de Estrategias**

Completa la siguiente tabla:

| Versión             | Directiva usada                    | Iteraciones y tiempo (s) |      |              | Observaciones |
|---------------------|------------------------------------|--------------------------|------|--------------|---------------|
| Secuencial          | Ninguna                            | euler                    | 2369 | 1.126667e-01 |               |
|                     |                                    | laplace                  | 2204 | 1.049998e-01 |               |
|                     |                                    | constanza                | 2040 | 9.683920e-02 |               |
|                     |                                    | fraccion                 | 2389 | 1.135288e-01 |               |
|                     |                                    | gaussian                 | 7609 | 3.613167e-01 |               |
| Paralelo básico     | Parallel for reduction             | euler                    | 2368 | 1.888211e-01 |               |
|                     |                                    | laplace                  | 2207 | 6.290498e-02 |               |
|                     |                                    | constanza                | 2042 | 7.264925e-02 |               |
|                     |                                    | fraccion                 | 2390 | 3.620198e-02 |               |
|                     |                                    | gaussian                 | 7621 | 1.313134e-01 |               |
| Colapsado de bucles | Parallel for collapse(2) reduction | euler                    | 2366 | 7.179466e-02 |               |
|                     |                                    | laplace                  | 2198 | 4.467217e-02 |               |
|                     |                                    | constanza                | 2028 | 7.213146e-02 |               |
|                     |                                    | fraccion                 | 2383 | 1.509673e-01 |               |
|                     |                                    | gaussian                 | 7619 | 1.565411e-01 |               |

|                                     |                                                       |           |      |              |  |
|-------------------------------------|-------------------------------------------------------|-----------|------|--------------|--|
| Inicialización y fuente en paralelo | Parallel sections firstprivate section                | euler     | 2402 | 1.156561e-01 |  |
|                                     |                                                       | laplace   | 2204 | 1.065159e-01 |  |
|                                     |                                                       | constanza | 2168 | 1.048587e-01 |  |
|                                     |                                                       | fraccion  | 2402 | 1.165256e-01 |  |
|                                     |                                                       | gaussian  | 1708 | 8.429295e-02 |  |
| Control explícito + schedule        | Parallel for reduction schedule (static)              | euler     | 2369 | 9.329464e-02 |  |
|                                     |                                                       | laplace   | 2206 | 3.210103e-01 |  |
|                                     |                                                       | constanza | 2044 | 2.101419e-01 |  |
|                                     |                                                       | fraccion  | 2392 | 1.972933e-01 |  |
|                                     |                                                       | gaussian  | 7629 | 8.628567e-01 |  |
| Control de sincronización           | Parallel single for nowait reduction critical barrier | euler     | 2369 | 2.794073e-01 |  |
|                                     |                                                       | laplace   | 2208 | 2.976169e-01 |  |
|                                     |                                                       | constanza | 2049 | 2.920471e-01 |  |
|                                     |                                                       | fraccion  | 2390 | 3.046331e-01 |  |
|                                     |                                                       | gaussian  | 7628 | 8.576414e-01 |  |

## Conclusiones

1. ¿Cuál fue la versión más rápida?

- Para Euler fue el colapsado de bucles con 7.179466e-02 segundos

- Para Laplace fue el colapsado de bucles con 4.467217e-02 segundos

- Para Constanza fue el colapsado de bucles con 7.213146e-02 segundos

- Para Fraccion fue el paralelo básico con 3.620198e-02 segundos

- Para Gaussian fue el inicialización y fuente en paralelo con 8.429295e-02 segundos

2. ¿Qué directiva es más fácil de usar?

Paralelo básico ya que la función de inicialización de la grilla y solve poisson son funciones rápidas que no necesitan de paralelización y solo se emplea en el ciclo for que requiere de mayor número de hilos.

3. ¿Hubo directivas que empeoraron el rendimiento? ¿Por qué?

Sí, task, sync, debido a la sincronización de parámetros y tareas asignadas en los ciclos en la inicialización de la grilla. Se tuvo que copiar variables que no tomaba la directiva firstprivate. Esto acrecentaba el cálculo y ocupación de memoria.

Actividad 6: Contar el número de iteraciones con acceso compartido

En esta actividad, vas a modificar la función solve\_poisson para contar el número total de iteraciones del bucle externo (las actualizaciones completas de la grilla) utilizando una variable global o compartida.

Instrucciones:

- Declara una variable `int iterations = 0;` como global o como variable compartida si usas una región paralela.
- Aumenta su valor cada vez que finaliza una iteración del bucle while ( $\text{delta} > \text{TOL}$ ).
- Utiliza una de las siguientes estrategias para evitar condiciones de carrera:
- Una sección crítica con `#pragma omp critical`
- Una operación atómica con `#pragma omp atomic`

## Conclusiones

- **¿Qué diferencias observas en el rendimiento entre usar critical y atomic?**

La directiva atomic es más eficiente en términos de rendimiento, ya que realiza la operación de manera atómica sobre una única variable, sin bloquear el acceso a otras partes del código. Solo se bloquea la operación específica en la variable que se está modificando. Esto permite una mayor concurrencia, ya que múltiples hilos pueden ejecutar operaciones en paralelo sin esperar a que otros terminen, siempre y cuando no estén modificando la misma variable atómica. Aunque también la directiva critical garantiza acceso exclusivo a la variable compartida, introduce más overhead. Esto se debe a que la sección crítica bloquea toda la sección de código, lo que significa que solo un hilo a la vez puede ejecutar esa sección, y los demás deben esperar a que termine el hilo actual.

- **¿En qué casos sería preferible una sobre la otra?**

**Atomic:**

- Si solo se desea modificar una única variable de forma sencilla.
- La operación que se realiza sobre la variable es ligera y no requiere de varias instrucciones complejas.
- Se busca minimizar el overhead de sincronización y mejorar el rendimiento en programas con alta concurrencia.

**Critical:**

- Se deben realizar operaciones complejas dentro de una sección crítica, como modificaciones de varias variables o cálculos que involucren más de una operación sobre la variable compartida.
- Hay múltiples variables compartidas que deben ser accedidas de manera exclusiva por cada hilo dentro de una región.
- La contención de hilos es baja o moderada, y no se espera un gran impacto en el rendimiento debido al overhead de la sincronización.

## Actividad 7: Exploración con tareas (task)

- Hasta ahora se ha usado paralelización por bucles. En esta actividad te proponemos experimentar con la directiva task de OpenMP para dividir el dominio en bloques y asignar tareas a distintos hilos.

### Instrucciones:

- Dentro de solve\_poisson, divide la grilla en bloques (por ejemplo, en cuadrantes o bloques de tamaño fijo).
- Dentro del bucle while, crea una región paralela y dentro de ella una sección single con múltiples task para cada bloque.
- Cada tarea deberá actualizar su bloque local de la grilla.
- Asegúrate de sincronizar con #pragma omp taskwait.

### Ejemplo simplificado:

```
#pragma omp parallel
{
    #pragma omp single
    {
        for (int bi = 1; bi < M; bi += block_size) {
            for (int bj = 1; bj < N; bj += block_size) {
                #pragma omp task
            }
        }
    }
}
```

```

        {
            for (int i = bi; i < std::min(bi + block_size, M);
++i) {
                for (int j = bj; j < std::min(bj + block_size, N);
++j) {
                    // actualización de T[i][j]
                }
            }
        }
    }
}
#pragma omp taskwait
}
}

```

### Conclusión:

¿Cómo se compara el rendimiento usando task con respecto a parallel for?  
 ¿Qué ventajas y desventajas tiene este enfoque?

El paralelismo por tareas tiene más overhead, pero es útil para trabajos irregulares. Control más fino sobre la distribución del trabajo. Mayor overhead que en bucles paralelos simples.

### COMO AJUSTAR Y ENTREGAR EL PROYECTO.

Este es un ejemplo de la estructura final para entregar.

```

Taller_OpenMP_Poisson/
├── Makefile
├── README.md
├── data/
│   └── solucion_serial.dat #Archivos de salida ... todos los de salida
├── src/
│   ├── poisson_serial.cpp      # Código base sin paralelismo
│   ├── poisson_parallel_for.cpp # Con parallel for
│   ├── poisson_critical.cpp    # Con sección crítica
│   ├── poisson_atomic.cpp     # Con variable atómica
│   ├── poisson_task.cpp       # Con tareas
│   └── utils.h                # Encabezado común (opcional)
├── actividades/
│   └── Informe de Resultados OpenMP_Ecuacion_Poisson.docx
└── imag/

```

```
|   └── Todas las imágenes conseguidas.
└── bin/
    ├── poisson_serial
    ├── poisson_parallel_for
    ├── poisson_critical
    ├── poisson_atomic
    └── poisson_task
```